MaquiMu — Plataforma de Gestión de Alquiler de Maquinaria Pesada

1. Descripción General

MaquiMu es una plataforma integral para la gestión de alquiler de maquinaria pesada, compuesta por tres aplicaciones que comparten un mismo backend:

Componente	Tecnología	Rol
Backend	Spring Boot 3.2 + Java	API REST centralizada
Frontend	Angular 17 + Bootstrap 5	Panel web para operarios y clientes
Mobile	Android (Kotlin)	App móvil exclusiva para clientes

---

2. Arquitectura

Backend — Arquitectura Hexagonal / Clean Architecture

El backend está organizado en un proyecto Gradle multi-módulo con tres capas:

```
maquimu-backend/
├── dominio/          → Modelos, puertos (interfaces) y servicios de dominio
├── aplicacion/       → Comandos, consultas, manejadores (casos de uso)
└── infraestructura/  → Controladores REST, JPA, seguridad, configuración
```

Flujo de una petición:

```
Cliente HTTP → Controlador → Manejador (caso de uso) → Servicio de dominio →
Puerto/Adaptador → Base de datos
```

Frontend — SPA con Lazy Loading

```
maquimu-frontend/src/app/
├── auth/       → Login, registro
├── admin/      → Dashboard, inventario, clientes, alquileres, facturación
├── client/     → Portal, solicitar alquiler, mis alquileres, mis facturas
├── core/       → Servicios, guards, interceptores, modelos
└── shared/     → Componentes reutilizables (navbar, sidebar, modales)
```

Mobile — MVVM con Retrofit

```
maquimu-mobile/app/src/main/java/com/sena/proyecto/
├── ui/           → Activities, Fragments, ViewModels
├── data/         → Repositorios, ApiService (Retrofit), modelos
└── utils/        → Constantes, sesión
```

---

3. Tecnologías Utilizadas

Capa	Stack
Backend	Java 17, Spring Boot 3.2, Spring Security, JWT, JPA/Hibernate, Flyway, MySQL, OpenPDF, JavaMail (SMTP)
Frontend	Angular 17, TypeScript, Bootstrap 5, Angular SSR, RxJS
Mobile	Kotlin, AndroidX, Material Components, Retrofit + OkHttp, Coroutines, LiveData/ViewModel, SharedPreferences
Base de datos	MySQL con migraciones Flyway (DDL + DML)

---

4. Modelo de Datos

```
┌──────────┐      ┌──────────┐      ┌──────────┐      ┌──────────┐
│ Usuario  │──────→│ Cliente  │──────→│ Alquiler │──────→│ Factura  │
│          │      │          │      │          │      │          │
│ rol      │      │ empresa  │      │ estado   │      │ total    │
│ estado   │      │ telefono │      │ fechas   │      │ estado   │
└──────────┘      │ direccion│      │ costo    │      │ pdf      │
                  └──────────┘      │ extensión│      └──────────┘
                                    └──────────┘
                                         │
                                         │
                                    ┌──────────┐
                                    │ Maquinaria│
                                    │ nombre   │
                                    │ tipo     │
                                    │ precio/día│
                                    │ estado   │
                                    └──────────┘
```

Estados clave:

- **Usuario:** `PENDIENTE_APROBACION` → `ACTIVO` / `RECHAZADO` / `RESTABLECER`
- **Maquinaria:** `DISPONIBLE` / `ALQUILADO` / `EN_MANTENIMIENTO`
- **Alquiler:** `PENDIENTE` → `APROBADO` → `ACTIVO` → `FINALIZADO` (con extensiones posibles)
- **Factura:** `PENDIENTE` → `PAGADO` / `ANULADO`

---

5. Funcionalidades por Rol

Operario (Admin Web)

Módulo	Funcionalidad
Dashboard	KPIs: alquileres activos, maquinaria disponible, ingresos
Clientes	Aprobar/rechazar registros, asignar contraseña temporal por email
Inventario	CRUD de maquinaria (nombre, tipo, precio, estado, imagen)
Alquileres	Aprobar, rechazar, entregar, finalizar, gestionar extensiones
Facturación	Generar facturas, cambiar estado de pago, descargar PDF

Cliente (Web + Mobile)

Módulo	Funcionalidad
Portal	KPIs personales, notificaciones
Solicitar alquiler	Seleccionar maquinaria disponible, definir fechas
Mis alquileres	Ver historial, solicitar extensión
Mis facturas	Ver facturas, descargar PDF
Perfil	Editar datos personales, cambiar contraseña

---

6. Seguridad

- **Autenticación:** JWT stateless con token Bearer en cada petición.
- **Hashing:** Bcrypt para contraseñas.
- **Endpoints públicos:** Solo `/auth/login`, `/auth/register` y `/auth/solicitar-restablecimiento`.
- **Guards (frontend):** `AuthGuard` verifica token activo; `RoleGuard` restringe rutas por rol.
- **Interceptor:** Inyecta automáticamente el header `Authorization` en cada petición HTTP.
- **Mobile:** Bloquea login de operarios; solo permite acceso a clientes.

---

7. API REST — Endpoints Principales

Recurso	Método	Ruta	Descripción
Auth	POST	`/auth/register`	Registro de cliente
Auth	POST	`/auth/login`	Inicio de sesión
Auth	POST	`/auth/solicitar-restablecimiento`	Solicitar reset de contraseña
Usuarios	GET	`/usuarios/pendientes`	Listar usuarios pendientes
Usuarios	PATCH	`/usuarios/{id}/aprobar`	Aprobar usuario

Recurso	Método	Ruta	Descripción
Usuarios	PATCH	`/usuarios/{id}/asignar-password`	Enviar contraseña temporal por email
Clientes	GET/POST/PUT/DELETE	`/clientes`	CRUD de clientes
Maquinaria	GET/POST/PUT/DELETE	`/maquinaria`	CRUD de maquinaria
Maquinaria	GET	`/maquinarias/disponibles`	Listar maquinaria disponible
Alquileres	POST	`/alquileres`	Crear solicitud de alquiler
Alquileres	PATCH	`/alquileres/{id}/aprobar`	Aprobar alquiler
Alquileres	PATCH	`/alquileres/{id}/finalizar`	Finalizar alquiler
Alquileres	POST	`/alquileres/{id}/extension`	Solicitar extensión
Facturas	POST	`/facturas`	Generar factura
Facturas	GET	`/facturas/{id}/pdf`	Descargar PDF de factura
Dashboard	GET	`/dashboard/operario`	KPIs del operario
Dashboard	GET	`/dashboard/cliente`	KPIs del cliente

> Base: `/api/maquimu/v1`

---

8. Pantallas de la Aplicación

Frontend Web

Ruta	Pantalla
`/auth/login`	Inicio de sesión
`/auth/register`	Registro de cliente
`/admin/dashboard`	Dashboard del operario
`/admin/inventory`	Gestión de inventario
`/admin/clients`	Gestión de clientes
`/admin/rentals`	Gestión de alquileres
`/admin/financial`	Gestión financiera
`/client/portal`	Portal del cliente
`/client/request-rental`	Solicitar alquiler
`/client/my-rentals`	Mis alquileres
`/client/my-invoices`	Mis facturas

App Móvil (Android)

Pantalla	Descripción
LoginActivity	Login + diálogo de recuperación de contraseña
RegisterActivity	Registro de cliente
DashboardFragment	KPIs + notificaciones
RentalsFragment	Lista de alquileres + detalle
InvoicesFragment	Lista de facturas + descarga PDF
ProfileFragment	Perfil, edición y cierre de sesión

---

9. Características Destacadas

- **Generación de PDF:** Facturas generadas con OpenPDF, descargables desde web y móvil.
- **Notificaciones en tiempo real:** Polling periódico que detecta nuevos alquileres, facturas, registros pendientes y cambios de estado.
- **Correo SMTP:** Envío de contraseñas temporales por email al restablecer acceso.
- **Migraciones de BD:** Flyway gestiona esquema (DDL) y datos iniciales (DML) de forma versionada.
- **SSR:** Angular Server-Side Rendering para mejor SEO y rendimiento inicial.
- **Validación de dominio:** Cálculo de costos y validación de disponibilidad en la capa de dominio.
- **Multiplataforma:** Un solo backend sirve tanto al frontend web como a la app móvil.

---

10. Flujo de Negocio Principal

```
1. Cliente se registra (web/móvil)
        ↓
2. Operario aprueba la cuenta
        ↓
3. Cliente inicia sesión y solicita alquiler
        ↓
4. Operario aprueba el alquiler
        ↓
5. Operario registra entrega de maquinaria → Alquiler ACTIVO
        ↓
6. (Opcional) Cliente solicita extensión → Operario aprueba/rechaza
        ↓
7. Operario finaliza el alquiler
        ↓
8. Operario genera factura → Cliente descarga PDF
        ↓
9. Operario registra pago → Factura PAGADA
```

---

11. Cómo Ejecutar

Backend

```
cd maquimu-backend
./gradlew bootRun
# Requiere MySQL en localhost:3306 con BD 'maquimu_db'
# Puerto: 8080
```

Frontend

```
cd maquimu-frontend
npm install
ng serve
# Puerto: 4200
```

Mobile

```
Abrir maquimu-mobile/ en Android Studio → Run en emulador o dispositivo
# Apunta a http://10.0.2.2:8080/api/ (emulador → localhost del host)
```

---

12. Patrones de Diseño Utilizados

Patrones Arquitectónicos

Patrón	Clasificación	Dónde se aplica
Arquitectura Hexagonal (Ports & Adapters)	Arquitectónico	Backend: `dominio/` define puertos (interfaces), `infraestructura/` implementa adaptadores (JPA, SMTP, PDF). El dominio no depende de frameworks.
Clean Architecture	Arquitectónico	Backend: las dependencias fluyen hacia adentro (infraestructura → aplicación → dominio). Cada capa tiene su propio módulo Gradle con dependencias explícitas.
CQRS (Command Query Responsibility Segregation)	Arquitectónico	Backend: separación explícita entre `comando/` (escritura) y `consulta/` (lectura) en la capa de aplicación. Cada operación tiene su propio manejador.
MVVM (Model-View-ViewModel)	Arquitectónico	Mobile: `ViewModel` expone datos vía `LiveData`, las Activities/Fragments observan cambios. La vista no accede directamente a los datos.
SPA (Single Page Application)	Arquitectónico	Frontend: Angular carga una sola página HTML y navega entre vistas sin recargar el navegador, usando el Router de Angular.

Patrones de Diseño (GoF y otros)

Patrón	Clasificación GoF	Dónde se aplica
Adapter	Estructural	Backend: cada adaptador en `infraestructura/` implementa un puerto del dominio. Ej: `RepositorioAlquilerMySql` adapta `PuertoRepositorioAlquiler`.
Factory	Creacional	Backend: clases `Fabrica*` en `aplicacion/` transforman comandos/DTOs en modelos de dominio. Ej: `FabricaAlquiler`, `FabricaMaquinaria`.
Repository	Acceso a datos	Backend: puertos como `PuertoRepositorioAlquiler` abstraen la persistencia. Mobile: `AuthRepository`, `RentalRepository` centralizan acceso a datos remotos.
Interceptor	Comportamiento	Backend: `JwtAuthenticationFilter` intercepta cada petición HTTP para validar el token. Frontend: `AuthInterceptor` inyecta el header JWT. Mobile: `AuthInterceptor` de OkHttp añade el token.
Observer	Comportamiento	Frontend: RxJS Observables para respuestas HTTP y estado reactivo. Mobile: LiveData notifica a las vistas cuando cambian los datos en el ViewModel.
Strategy	Comportamiento	Backend: servicios de dominio como `ServicioCalculadorCostoAlquiler` y `ServicioValidadorDisponibilidadMaquinaria` encapsulan algoritmos intercambiables de negocio.
Singleton	Creacional	Backend: Spring gestiona beans como singletons por defecto. Mobile: `RetrofitClient` usa `object` de Kotlin (singleton). Frontend: servicios Angular son singleton por defecto con `providedIn: 'root'`.
Guard	Comportamiento	Frontend: `AuthGuard` y `RoleGuard` implementan `CanActivate` para proteger rutas según autenticación y rol.
DTO (Data Transfer Object)	Transferencia	Backend: comandos y consultas en `aplicacion/` actúan como DTOs entre controlador y manejador, desacoplando la API de los modelos de dominio.
Lazy Loading	Rendimiento	Frontend: los módulos `admin/`, `client/` y `auth/` se cargan bajo demanda usando `loadChildren` en el Router de Angular.

---

13. Glosario Técnico

API REST

Application Programming Interface — Representational State Transfer. Estilo de arquitectura para servicios web que usa métodos HTTP (GET, POST, PUT, PATCH, DELETE) sobre recursos identificados por URLs.

Permite comunicación stateless entre cliente y servidor.

Spring Boot

Framework de Java que simplifica la creación de aplicaciones Spring con configuración automática, servidor embebido (Tomcat) y gestión de dependencias. Versión utilizada: 3.2.

Spring Security

Módulo de Spring para autenticación y autorización. En este proyecto configura filtros de seguridad HTTP, protección de endpoints y manejo de sesiones stateless con JWT.

JWT (JSON Web Token)

Estándar (RFC 7519) para transmitir información entre partes como un objeto JSON firmado. Se usa como token de autenticación: el servidor lo genera al hacer login y el cliente lo envía en cada petición posterior.

JPA / Hibernate

Java Persistence API: especificación estándar de Java para mapeo objeto-relacional (ORM). **Hibernate** es su implementación más popular. Permite interactuar con la base de datos usando objetos Java en lugar de SQL directo.

Flyway

Herramienta de migración de bases de datos. Versiona los cambios de esquema con archivos SQL numerados (`V1__`, `V2__`...) y los aplica de forma ordenada y reproducible.

MySQL

Sistema de gestión de bases de datos relacional de código abierto. Almacena toda la información del sistema: usuarios, clientes, maquinaria, alquileres y facturas.

OpenPDF

Biblioteca Java para generar documentos PDF programáticamente. Se usa para crear las facturas descargables del sistema.

SMTP (Simple Mail Transfer Protocol)

Protocolo estándar para envío de correos electrónicos. Se usa con JavaMail para enviar contraseñas temporales cuando un operario restablece la cuenta de un cliente.

Bcrypt

Algoritmo de hashing adaptativo para contraseñas. Incluye sal automática y factor de costo configurable, lo que lo hace resistente a ataques de fuerza bruta.

Gradle

Herramienta de automatización de compilación. Gestiona dependencias, compilación y empaquetado. Soporta proyectos multi-módulo como el backend y la app móvil.

Angular

Framework de desarrollo web frontend creado por Google. Usa TypeScript, componentes declarativos y un sistema de inyección de dependencias. Versión utilizada: 17 con standalone components.

TypeScript

Superconjunto de JavaScript con tipado estático. Detecta errores en tiempo de desarrollo y mejora el autocompletado. Angular está construido sobre TypeScript.

Bootstrap

Framework CSS para diseño responsive. Proporciona componentes prediseñados (botones, tablas, modales, grids) que aceleran el desarrollo de interfaces. Versión: 5.

RxJS (Reactive Extensions for JavaScript)

Biblioteca para programación reactiva con Observables. Permite manejar flujos de datos asíncronos (peticiones HTTP, eventos de usuario) de forma declarativa. Es parte fundamental de Angular.

SSR (Server-Side Rendering)

Técnica donde el HTML se genera en el servidor antes de enviarse al navegador. Mejora el SEO y el tiempo de carga inicial. Se implementa con Angular Universal.

Kotlin

Lenguaje de programación moderno para la JVM, oficialmente recomendado por Google para desarrollo Android. Más conciso y seguro que Java, con soporte nativo para null-safety y coroutines.

Retrofit

Biblioteca de Square para Android/Kotlin que convierte una API REST en una interfaz de Kotlin. Genera automáticamente las peticiones HTTP a partir de anotaciones.

OkHttp

Cliente HTTP eficiente para Java/Kotlin. Retrofit lo usa internamente. Soporta interceptores para modificar peticiones (ej: añadir headers de autenticación).

Coroutines

Mecanismo de Kotlin para programación asíncrona. Permiten escribir código secuencial que se ejecuta sin bloquear el hilo principal, ideal para operaciones de red en Android.

LiveData

Clase observable del ciclo de vida de Android. Notifica automáticamente a las vistas cuando los datos cambian y respeta el ciclo de vida del componente (Activity/Fragment).

ViewModel

Componente de Android Architecture Components que almacena datos relacionados con la UI. Sobrevive a cambios de configuración (como rotaciones de pantalla).

SharedPreferences

Almacenamiento clave-valor persistente en Android. Se usa para guardar el token JWT y datos de sesión del usuario localmente en el dispositivo.

CRUD

Create, Read, Update, Delete. Las cuatro operaciones básicas sobre datos persistentes. En una API REST se mapean a POST, GET, PUT/PATCH y DELETE.

KPI (Key Performance Indicator)

Indicador clave de rendimiento. Los dashboards del sistema muestran KPIs como: alquileres activos, maquinaria disponible, ingresos totales y facturas pendientes.

Polling

Técnica donde el cliente consulta periódicamente al servidor para verificar si hay datos nuevos. El frontend y la app móvil usan polling para detectar notificaciones.

DDL / DML

Data Definition Language: sentencias SQL que definen estructura (CREATE TABLE, ALTER TABLE). **Data Manipulation Language:** sentencias que manipulan datos (INSERT, UPDATE, DELETE). Flyway separa las migraciones en carpetas `ddl/` y `dml/`.

DTO (Data Transfer Object)

Objeto simple para transferir datos entre capas. Evita exponer los modelos de dominio directamente en la API. En este proyecto, los comandos y consultas de la capa de aplicación cumplen este rol.